Multi-Dimensional Autoscaling of Stream Processing Services on Edge Devices

Boris Sedlak, *Member, IEEE*, Philipp Raith, *Member, IEEE*, Andrea Morichetta, Víctor Casamayor Pujol, *Member, IEEE*, and Schahram Dustdar, *Fellow, IEEE*

***Abstract*—Edge devices have limited resources, which inevitably leads to situations where local resources do not suffice to serve the changing request load. While existing autoscaling mechanisms focus entirely on resource scaling, Edge devices require alternative ways to sustain Service Level Objectives (SLOs) of deployed services. To address these issues, we introduce a Multi-dimensional Autoscaling Platform (MUDAP) that supports fine-grained vertical scaling across service- and resource-level dimensions. MUDAP supports service-specific scaling tailored to available parameters, e.g., scale data quality or model size for a particular service. To optimize service execution on Edge devices, where multiple services compete for one global resource pool, we present a scaling agent using Regression Analysis of Structural Knowledge (RASK). The RASK agent efficiently learns a continuous regression model of the processing environment for optimizing the global SLO fulfillment across the deployed services. We compared our approach with two autoscalers—the Kubernetes VPA and a reinforcement learning agent—for scaling up to 9 services on a single Edge device. Our results showed that RASK can infer an accurate regression model in merely 20 iterations (i.e., observe 200 s of processing). By increasingly adding elasticity dimensions, RASK sustained the highest request load with 28% fewer SLO violations, compared to baselines.**

***Index Terms*—Autoscaling, service level objectives, elasticity, edge computing, distributed systems, regression analysis.**

I. Introduction

THE Edge layer has become a pillar [1] for services that demand low-latency, high-reliability access to computing, such as in automotive [2] or disaster response [3] scenarios. In such dynamic and ever-changing scenarios, optimizing the resource allocation and ensuring performance guarantees—quantified through Service Level Objectives (SLOs)—is paramount. To assign processing services the desired resources under changing conditions (e.g., fluctuating demand), elastic computing principles [4] and autoscaling policies provide a remedy. In this context, the Kubernetes Vertical Pod Autoscaler (VPA), as introduced in 2023 [5], offers fast resource adaptation and service elasticity—essential for resource and time-sensitive scenarios. Albeit serverless computing emerges as a solution for flexible application management, it still suffers from performance and resource management issues [6], [7], especially during request peaks. Hence, using dependable, lightweight containers [8] remains the most reliable solution.

To ensure high-level SLOs, like efficiency or accuracy [9], applications need to adjust and optimize a myriad of lower-level components. Yet, traditional autoscalers, including the Kubernetes VPA, have two major limitations: First, they often use simple rule-based decision-making, and its effectiveness highly depends on the operator to manually fine-tune it. Therefore, researchers and practitioners have advocated learning-based solutions—prominently using Reinforcement Learning (RL) [10], [11], [12], [13], [14], [15]. Still, these approaches come with other issues, including long, sample-inefficient training periods, and limited interpretability of inferred actions. Second, to fulfill SLOs, traditional autoscalers are limited to claiming additional resources—hence, they operate only in one *elasticity dimension*. While this simplifies service orchestration, it does not exploit the manifold of potential elasticity strategies across different services. In particular, video inference services might dynamically scale the service quality [16] or model size [17]; despite the huge impact of these parameters on the service throughput, they are not commonly used in autoscaling. Notably, individual Edge devices can only provide resources up to a strict limit. To ensure processing SLOs regardless, computation is commonly redirected through offloading [18] or horizontal autoscaling [19]. However, emerging architectures like the Computing Continuum [20] face volatile networking and resource availability; this demands additional, decentralized mechanisms to optimize SLO fulfillment.

While horizontal autoscaling can also be applied within a single node—deploying multiple replicas of a service to leverage additional CPU cores—its efficiency heavily depends on how a service is implemented. In this paper, we focus on an isolated Edge deployment environment and specifically address services that natively benefit more from adding additional

Received 29 July 2025; revised 2 July 2026; accepted 4 July 2026. Date of publication 8 July 2026; date of current version 9 October 2026. This work was supported in part by European Union's Horizon Europe research and innovation program under Grant 101135576 (INTEND) and Grant 101070186 (TEADAL), in part by CNS2023-144359, in part by the European Union NextGenerationEU/PRTR under Grant MICIU/AEI/10.13039/501100011033, and in part by Open Access Funding Programme supported by TU Wien Bibliothek. *(Corresponding author: Boris Sedlak.)*

Boris Sedlak, Philipp Raith, and Andrea Morichetta are with Distributed Systems Group, TU Wien, 1040 Vienna, Austria (e-mail: b.sedlak@dsg.tuwien.ac.at; p.raith@dsg.tuwien.ac.at; a.morichetta@dsg.tuwien.ac.at).

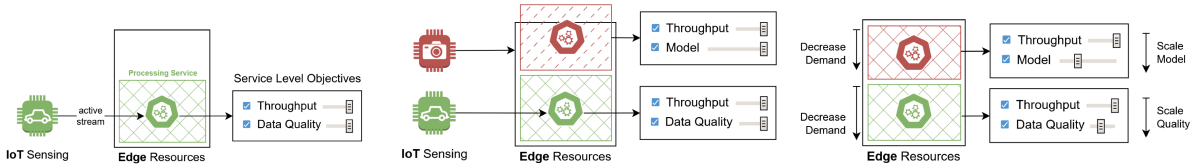
Víctor Casamayor Pujol is with the Department of Engineering, Universitat Pompeu Fabra, 08005 Barcelona, Spain (e-mail: v.casamayor@upf.edu).

Schahram Dustdar is with Distributed Systems Group, TU Wien, 1040 Vienna, Austria, also with the Department of Engineering, Universitat Pompeu Fabra, 08005 Barcelona, Spain, and also with ICREA, 08010 Barcelona, Spain (e-mail: schahram.dustdar@upf.edu).

This article has supplementary downloadable material available at https://doi.org/10.1109/TSC.2026.3711142, provided by the authors.

Digital Object Identifier 10.1109/TSC.2026.3711142

© 2026 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/



(a) An IoT stream is processed at the closest Edge device. The processing service has enough resources allocated to fulfill its two SLOs.

(b) Another IoT device wishes to process sensor data at the Edge device. However, to fulfill its SLOs, it requires more resources than available.

(c) To accommodate more processing services at the Edge device, the services can decrease the provided quality and the respective resource demand.

Fig. 1. Co-locating multiple processing services on a constrained Edge device by accurately trading off service quality.

resource capacity to a single, multi-threaded instance rather than creating multiple low-resource replicas.

Our work addresses this gap by offering a flexible, *multi-dimensional* autoscaling platform—controlling both the infrastructure and the application [21]—for harmonizing the SLO fulfillment on resource-constrained Edge devices. Conceptually, we first analyze the behavior and SLO fulfillment of all deployed services through a regression model, and then optimize the device-wide SLO fulfillment through a numerical solver. The numerical solver provides fine-grained assignments for vertically scaling both service- and resource-level parameters within an Edge node. By adding the control at the device level, we ensure that scaling actions are harmonious and do not penalize other services. We implement our solution *service-agnostic*, making it modular for adding new service types or elasticity parameters. Likewise, our methodology can cope with changing load patterns or SLO thresholds and ensure its model accuracy despite distribution shifts.

An effective way to illustrate the impact of our approach is through a key scenario in Fig. 1: Sensor data from different sources (vehicles, cameras, etc.) is processed by services on an Edge device. This is a time-sensitive, dynamic environment where suboptimal decisions can lead to dangerous outcomes. However, when multiple services have to operate on the same node (see Fig. 1(b)), it is not possible to meet their resource demands, creating conflicts and mitigating performance. In this case, downscaling the allocated resources for an object detection model can impact its reactivity or, in the worst case, break its execution. For this reason, our approach enables multi-dimensional elasticity control over services. This solution allows trading off qualitative aspects, like data quality or model size, for sustaining essential functionality, like throughput. This approach—also known as *brownout* [22]—is downgrading the service quality to avoid resource saturation. In our case, we tailor scaling actions to the individual characteristics of processing services, enabling optimized configurations and SLO fulfillment across both resource and service dimensions. At the same time, our solver ensures fair resource allocation to guarantee the performance of all concurrent services. Overall, we contribute to advance the state-of-the-art through:

- MUDAP, a Multi-dimensional Autoscaling Platform for resource-constrained Edge environments that supports fine-grained vertical scaling of elasticity parameters without the need to restart processing containers. It allows tailor-made scaling for heterogeneous services by exposing particular service- or resource parameters.
- RASK, a regression-based scaling agent that optimizes the SLO fulfillment across multiple competing processing services. RASK creates an explainable regression model that allows it to infer optimal scaling decisions.
- An extensive evaluation that highlights superior SLO fulfillment of multi-dimensional autoscaling under dynamic conditions. We compared RASK with existing autoscalers, including the Kubernetes VPA, and demonstrated how RASK sustains periods of high load with 28% fewer SLO violations, while barely introducing any CPU overhead. To achieve this performance, RASK was extremely sample-efficient, requiring merely 20 training iterations—corresponding to 200 s of processing.

The remainder of the paper is structured as follows. Section II introduces the core challenges and background of this work. Section III presents our MUDAP architecture and its main components, whereas in Section IV we introduce our RASK agent. Section V offers an in-depth evaluation of our proposed approach in comparison to reference baselines. Finally, in Section VI we outline the implications of our work, while Section VII concludes the paper.

II. Preliminaries

A. Motivating Example

Consider a smart city that consists of numerous IoT sensors and processing infrastructure distributed throughout a wide area. To optimize traffic flow and reduce emissions, we want to process various types of data streams (e.g., audio, video, temperature) through a series of containerized services. Consider Fig. 2, where an IoT device, like an IP camera, generates a continuous stream of video frames. To detect objects within the stream, e.g., cars in traffic junctions, video frames are processed by an Edge device in close vicinity.

Frames are collected in a standalone buffer, from where services consume them; this provides two benefits: (1) new data sources, like another camera, can simply stream to the same buffer; and (2) if the amount of generated data increases, a processing service can react to this according to the backpressure. From a service perspective, these changes only affect the buffer size. Therefore, we abstract the actual data sources as *requests per second* (RPS) received by the buffer. To sustain SLO fulfillment under varying RPS, services are scaled dynamically. While conventional scaling mechanisms, like horizontal autoscaling [19], often assume that workloads can be distributed

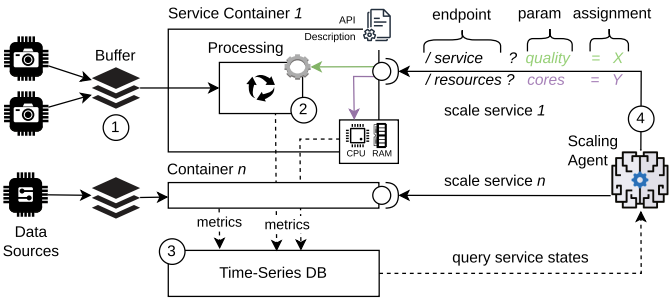


Fig. 2. Conceptual architecture of the MUDAP platform: ① IoT devices continuously ingest data into a buffer; ② data is collected and processed every second; ③ container and service metrics are scraped by a time-series DB; ④ the scaling agent queries the service states and adjusts the service execution and resource limits through the exposed API endpoints.

across a cluster of machines or multiple local service replicas, we narrow our scope to optimize isolated Edge devices with strict resource constraints. Within this scope, we focus on multi-dimensional vertical scaling for highly parameterizable and multi-threaded stream applications, where altering instance resources and application parameters provides finer control than container replication.

B. Core Challenges

Considering this example, we formulate three challenges that address (*C1*) SLO fulfillment on resource-constrained devices, (*C2*) multi-dimensional autoscaling for Edge devices and (*C3*) achieving SLO targets under dynamic workloads.

- *Challenge 1:* Vertical scaling can increase resource efficiency and SLO fulfillment [23]. However, scaling multiple services on an Edge device is challenging because services share limited resources. Typical approaches to vertical scaling, such as CPUs, thus have limited impact for improving service throughput. Service-aware vertical scaling, such as input size, could mitigate this issue and trade off quality in exchange for improved throughput.
- *Challenge 2:* Vertical scaling approaches must be tailored towards services, their particular resource needs, and the processing device. The impact of vertical scaling differs across services: while an additional CPU core or lower data quality might increase the throughput of one service, it might not affect another service type at all. Learning the impact of scaling actions per service is challenging and exacerbated in increasingly larger action spaces.
- *Challenge 3:* Fluctuating usage pattern mean the amount of IoT data—or IoT data sources—changes throughout the day. To keep SLOs fulfilled, while avoiding the risk of overprovisioning resources, the allocated resources must be adjusted. This is especially important in scenarios where multiple services on an edge device receive bursts of requests at the same time, calling for actions that optimize latency or throughput with resources at hand.

C. Background & Related Work

In the following, we describe two concepts that are central to our work and processing in general: SLOs and elasticity strategies. We reflect on their current state-of-the-art and present how they are used and extended in this work.

1) Service Level Objectives: In their classical sense [24], [25], SLOs track basic functional service aspects like availability or latency. In their wider sense, SLOs are increasingly used to track composed metrics that express high-level goals like effectiveness [9] or accuracy [26]—possibly supported through an SLO language [27] or orchestration framework [28]. While these approaches are arguably still immature, they allow consumers to freely express requirements, rather than choose between predefined SLOs. Hence, in the context of this paper, we adopt the *wider sense of SLOs* [29] for tracking functional aspects (e.g., latency or throughput) and non-functional aspects (e.g., video quality or model accuracy) alike.

Generally, an SLO q relates a variable to a target value t_q; for example, keeping service throughput $(tp) \geq 30$. Given a metrics $(m \in M)$ and an SLOs $(q \in Q)$, we calculate the SLO fulfillment (ϕ)[1] as a continuous value through

$$\phi(q, m) = \begin{cases} \frac{m}{t_q} & \text{if } m < t_q \\ 1.0 & \text{if } m \geq t_q \end{cases} \tag{1}$$

Under this representation, SLOs cannot be overfulfilled; hence, two metrics $m_{tp} = 40$ and $m_{tp} = 100$ would both achieve the maximum SLO fulfillment of $\phi = 1.0$.

2) Elasticity Strategies: The emergence of Cloud computing [30] supported dynamic changes to provisioned resources. Autoscaling platforms, like Kubernetes (k8s), use this for horizontal and vertical autoscaling. This dimension for flexible resource management, known as *elasticity*, has led to advanced control strategies. Recent work used this extensively for building RL-based solutions [10], [11], [12], [13], [14], [15]: given a few boundaries, the algorithm can learn the best strategy to dynamically adapt to application or infrastructure changes. Still, RL requires extensive training, accurate fine-tuning, and struggles with high-dimensional environments; hence, its use in production [13], [21] is still limited. Furthermore, while elasticity—as defined by [31]—is not limited to resource scaling, scaling other dimensions (e.g., quality) has not found much traction. We hypothesize that the Cloud's abundance of resources made it obsolete to compromise quality. This led to a situation where contemporary research [19], [32], [33], [34] on autoscaling focuses almost exclusively on adjusting or scheduling resources. Even prominent multi-dimensional approaches, like AWARE [12], focus solely on scaling resource demands—although they combine vertical *and* horizontal autoscaling.

The shift toward Edge computing exposes the limitations of traditional orchestration [35], [36] in managing constrained, heterogeneous resources. While collaboration strategies like offloading [37], [38] and the Computing Continuum [20], [39] provide relief, they primarily displace the problem by assuming remote availability. To ensure SLO fulfillment in truly isolated

[1] We choose the letter 'ϕ' due to its sound: SLO ful-*phi*-llment.

or resource-scarce environments, scaling the service quality becomes the last remedy.

While there exist works that provide elastic service quality, they are often named differently: model-less inference [17] dynamically chooses the size of ML models according to SLOs and available hardware; other options to ensure SLOs are filtering the video content [40] or changing stream parameters [16], [41]—like video resolution or frame rate. However, these works, in turn, do not support resource scaling.

While recent work explores simultaneous scaling of resources and quality [42], [43], these approaches remain impractical for several reasons. First, they tightly couple elasticity strategies with autoscalers—thus lacking a modular mechanism to register service-specific strategies or custom autoscaling algorithms. Second, selecting among elasticity strategies introduces an optimization problem that is addressed using model-free reinforcement learning. Such methods (e.g., Deep Q Networks) typically require thousands of iterations to converge. However, autoscaling actions commonly require multiple seconds to show effects [42], slowing down every single iteration of the agent. Finally, these approaches operate in a coarse-grained, discrete action space, which can prevent the system from reaching the global optimum.

Given that, we conclude that there is a clear gap for (1) an extensible multi-dimensional scaling platform that combines quality and resource scaling, while presenting clear interfaces for coupling different autoscalers; also (2) multi-dimensional autoscalers have not applied sample-efficient learning that allow (3) fine-grained adjustments of service-specific parameters. In the following, we address this through our modular autoscaling platform and regression-based scaling agent.

III. Multi-Dimensional Autoscaling Platform

Common autoscaling platforms are limited to resource scaling and hence, do not offer interfaces for coupling multi-dimensional scaling agents. To that extent, this section describes a Multi-dimensional Autoscaling Platform (MUDAP) that exposes an API for dynamically scaling both service and resource parameters of containerized processing services in an Edge device. In Fig. 2 we summarize the conceptual architecture of MUDAP, which we explain in the following. Afterwards, in Section IV, we present a scaling agent that uses MUDAP's interfaces for optimizing SLO fulfillment.

A. Processing Service Execution

To allow fine-granular management of allocated resources, each processing service is wrapped in an individual container. Every second, a time-series DB scapes containers' resource utilization and service-specific metrics. To address a service container, scaling agents provide the executing $host$ (or device), its service $type$, and container name. We summarize this through $s = \langle host, type, c_name \rangle$. Thus, service containers can be accessed by local or remote scaling agents equally.

TABLE I
Syntax for Defining Multiple Elasticity Strategies Per Service; Right Column Shown an Example Configuration

Structural Entry	Example Value
service type : list[str]	"obj-detector"
> elasticity str. : list[str]	"resources"
> url endpoint : str	"/resources"
> query parameter : list[str]	"cores"
> minimum value : float	1.0
> maximum value : float	8.0

B. Elastic Service Adaptation

By design, our multi-dimensional autoscaling platform can scale any parameter that can be dynamically adjusted during runtime, i.e., without container restart. We broadly categorize parameters into: resource constraints and service configurations. Resource constraints limit the allocated resources per service container, such as the maximum scheduled CPU quota or RAM. Service configurations adjust application-specific logic or functionality. For instance, a video detection service could adjust the size of a DNN model, or the size of the input tensor (i.e., the video resolution). Combined, these two sets form a service's *elasticity parameters*, which can be adjusted through a REST API hosted within each container. Fig. 2 shows the structure of such requests: a service endpoint, the elasticity parameter, and the parameter assignment. Thus, to change the input resolution of a video detection service, a potential API call could be */quality?resolution=1080*. This request is forwarded to the processing service; queries adjusting the CPU quota would go to the container. These changes do not require any container or application restart.

To allow scaling agents interact with the elasticity parameters, services offer an API description; Table I shows the respective syntax. For a specific service type, like "obj-detector", we define a list of elasticity strategies and their respective parameters. For each parameter, we specify the minimum and maximum value; assume a device could allocate up to 8 CPU cores (i.e., a CPU quota of 800.000.000 ns), the range would be $[1.0, 8.0]$. If the assignment exceeds the valid bounds, the value is clipped to the next valid assignment.

At the later evaluation of MUDAP, we implement the different endpoints in the service logic and manually define the API description for the different services.[2]

IV. Scaling Agent Design

While there exist various potential algorithms for implementing multi-dimensional scaling, we advocate model-based methods that allow interpreting the inferred scaling actions [41], [44]. Further, given the dynamic and fast-paced nature of IoT streams, building scaling policies with few samples is desirable. To that extent, we present a tailor-made scaling agent that applies Regression Analysis of Structural Knowledge (RASK). RASK continuously improves its structural knowledge of the environment—expressed through continuous variable relations,

[2] The interested reader can find the full API description here.

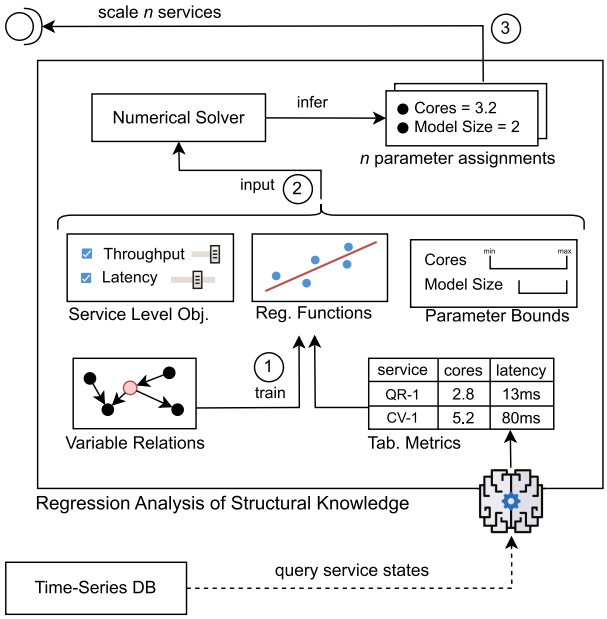


Fig. 3. Conceptual sequence of RASK: ① create a tabular structure from time-series data and train regression functions; ② supply functions, SLOs, and parameter bounds to numerical solver; ③ optimize parameter assignments for all monitored services and autoscaled through MUDAP API.

which it uses to repeatedly optimize the SLO fulfillment through a numerical solver. As depicted in Fig. 3, RASK first ① creates a tabular structure from the time-series data and trains the regression functions; it then ② supplies these functions together with SLOs and parameter bounds to the numerical solver. Finally, ③ the solver produces parameter assignments for all monitored services, which are scaled through the MUDAP API. These steps—the *RASK logic*—will be explained in the next subsections.

The RASK logic is executed in a continuous action-perception cycle—creating the *RASK agent*. However, how often should this autoscaling cycle be executed? An argument for slower cycles would be the execution overhead, whereas faster cycles might help during highly dynamic request patterns [45]. Also, changes to the processing environment are not immediately reflected, i.e., after an elasticity strategy, the system requires time to stabilize within the new configuration. For example, the Kubernetes (k8s) Vertical Pod Autoscaler[3](VPA) by default recommends scaling actions every 1 min; after applying an action, a 10 min cooldown period is enforced. However, in our initial experiments, processing services (e.g., object detection) stabilized in less than 5 s. This deserves dedicated optimization and analysis, but not in the scope of this paper. Hence, we simplify the decision and choose evaluation cycles of 10 s—detached from any processing service.

[3] Kubernetes VPA Description, last accessed on July 17, 2026.

A. Observation of Processing Environment

To collect the current state of the processing environment, the agent uses metrics from the time-series DB. Common time-series DBs (e.g., Prometheus) support metrics aggregation for stabilizing service states against natural fluctuations in the environment. While we set the evaluation interval to 10 s, we also discussed that scaling actions take up to 5 s for fully taking place; hence, we query a time series of the remaining 5 s and consider the average. We repeatedly capture this information to create the training data (D) for the regression functions. The agent's architecture thus favors fast paced adaptation cycles over a longer prediction horizon that uses more traces.

B. Regression Analysis of Structural Knowledge

Building continuous variables relations allows interpolating between few samples captured through controlled interventions [41]. However, compared to conventional autoscalers [32], it also allows inferring continuous scaling actions—promoting fine-grained scaling decisions that converge towards a global optima [46]. In its core, RASK uses a simple numerical solver for finding an optimal parameter assignment. The novelty, however, lies in turning multiple elasticity parameters into convex functions, which can then be passed to the solver. This allows estimating the impact of different elasticity strategies and the respective SLO fulfillment. Hence, the crux for inferring satisfying assignments is developing an accurate but generalized regression model.

1) Obtaining Structural Knowledge: Creating the regression model and expanding it towards unseen parameter combinations faces an exploration-exploitation tradeoff: should the agent explore to improve its understanding of the environment (i.e., the accuracy of regression functions), or should it exploit known assignments to quickly rise SLO fulfillment, but potentially end up with a suboptimal configuration. To that extent, RASK supports two hyperparameters for tuning the exploration: ξ—the length of an initial exploration phase, and η—a percentage of Gaussian noise added to inferred parameter assignments, commonly used for RL-based exploration. Section V analyzes the effects of both parameters.

So how do we know which variables to relate? As there is rich literature on developing this knowledge through structural learning [41], [47], [48], [49], we argue that existing work already addressed this research question. Hence, in the context of this paper, we supply directed variable relations—the structural knowledge (K)—according to expert knowledge. For a relation $k \in K$, which represents a variable's dependence on one or multiple other variables, we can extract the features (X) and target (Y) from the training data (D). As shown in (2), we then fit a regression function ($\mathbf{w}^*$) to the data. According to the relation, the polynomial degree (δ) can be customized—transforming the features X into a higher order space. For example, $\delta = 2$ produces a quadratic function.

$$\mathbf{w}^*(X, Y, \delta) := \arg\min_{\mathbf{w}} \sum_{i=1}^{D} \left(y_i - \mathbf{w}^\top \delta(x_i)\right)^2 \tag{2}$$

Algorithm 1: Regression Analysis of Structural Knowledge.

Require: services (S), training data (D), Gauss. noise (η), exploration (ξ), polynomial degree (δ), knowl. structure (K), parameter bounds (P), constraints (C), SLOs (Q)
Ensure: A {assignments for elastic parameters}
1: $W \leftarrow \varnothing$
2: rounds $\leftarrow$ (rounds $+ 1$) **if** defined, **else** 0
3: **if** rounds $< \xi$ **then**
4: **return** RAND_PARAM $(P, c_{\max})$
5: **end if**
6: **for** each $s \in S$ and $k \in K_s$ **do**
7: $X; Y \leftarrow D_s[k]$ {extract target features}
8: $W_s = \mathbf{w}^*(X, Y, \delta)$ {polynomial regression}
9: **end for**
10: $A' \leftarrow$ SOLVE (S, P, Q, W, C)
11: $A \leftarrow [\,$ NOISE$(a', \eta) \mid a' \in A'\,]$
12: **return** A

2) Performing Regression Analysis: To optimize the service execution, the RASK agent adjusts the exposed elasticity parameters through a numerical solver. To that extent, we collect the parameter bounds (P) using the scaling platform's API description (see Table I), a set of SLOs (Q) that define the desires QoE, and finally, the global resource constraints (C) governing the host. Using the numerical solver, the agent produces assignment (A) for each parameter ($p \in P$). For example, for a parameter $cores \in P$ with bounds $[1.0, 8.0]$, a valid assignment $a \in A$ would be $cores = 4.5$.

For a set of services (S), we now perform all steps of RASK as shown in Algorithm 1: the agent starts counting the number of autoscaling cycles (i.e., the *rounds*) and keeps exploring as long as $rounds < \xi$ stays true (Lines 2–4). Exploring, as shown by RAND_PARAM in (3), means randomly assigning all parameters according to a uniform distribution, but within their bounds and global resource constraints. While this assumes zero prior knowledge, future work could benefit from more sophisticated exploration, e.g., avoid already visited states.

$$\begin{aligned} \texttt{RAND_PARAM} := \;& \text{Draw } A \sim \mathcal{U}\left(p^{\min}, p^{\max}\right) \\ \text{s.t. } & \sum_{i=1}^{S} p_i \leq C_p \\ & p^{\min} \leq p \leq p^{\max} \quad \forall p \in P_i \end{aligned} \tag{3}$$

Once past the initial exploration period, the agent starts preparing the regression model (W)—a mere collection of all regression functions. For each service $s \in S$ and its structural relation $k \in K_s$, the agent does fit a regression function (Lines 6–9). In Line 10, the regression model W is then passed to the numerical solver, together with the parameter bounds (P), the SLOs (Q), and the global constraints (C). As shown in (4), the objective is to maximize the SLO fulfillment (ϕ) for all SLOs ($q \in Q$) by assigning parameters within the bounds and global constraints. For SLOs concerning dependent variables, we estimate their value according to the regression model—$\mathbf{w}_i(p_i)$. Using gradient descent, a numerical solver (e.g., SLSQP [50]) can optimize this objective.

$$\begin{aligned} \texttt{SOLVE} := \max_{A} \;& \sum_{i=1}^{S} \sum_{j=1}^{Q_i} \phi(q_j,\; p_i \wedge \mathbf{w}_i(p_i)) \\ \text{s.t. } & \sum_{i=1}^{S} p_i \leq C_p \\ & p^{\min} \leq p \leq p^{\max} \quad \forall p \in P_i \end{aligned} \tag{4}$$

Finally, we apply Gaussian noise to each assignment. As shown in (5), it first computes a standard deviation (σ) according to the assignment (a') and the noise ratio (η); for example, cores $= 4$ and $\eta = 0.1$ produce $\sigma = 0.4$. Each assignment is then shifted by its relative noise (or offset o).

$$\texttt{NOISE}\,(a, \eta) := a + o;\; o \sim \mathcal{N}(0, \sigma) \text{ and } \sigma = (a \times \eta)^2 \tag{5}$$

After reaching Line 12, the noisy parameter assignment is returned. Otherwise, if the agent was still exploring, Line 5 already returned the randomized assignment.

3) Tuning the Numerical Solver: A common choice is to start a numerical solver from a randomized, or averaged parameter assignment. Using this strategy, our initial experiments showed outliers in the duration of the solver. To minimize the risk of exceeding the autoscaling interval, we implement a caching mechanism: instead of starting the numerical solver from a random or default state, we cache the last parameter assignment to kickstart the solver. However, we hypothesize that this might trap the solver in a local optima, hence, we analyze the implications of the caching in Section V-C1.

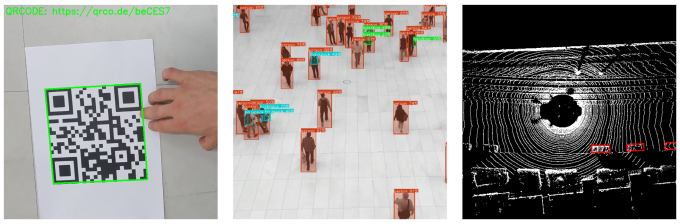
C. Service Autoscaling

Given the assignment (A) provided by RASK, the agent now has to scale the services. For this, it uses the API of the autoscaling platform to adjust the services according to the parameter assignments. For each assignment $a \in A$, this means sending a request to the exposed REST API; the structure of these requests was explained alongside Fig. 2.

This concludes the autoscaling cycle of the RASK agent, which is continuously executed every 10 seconds—the evaluation interval. However, as the upcoming evaluation also shows, the agent usually finished the autoscaling in a fraction of that time. Hence, it minimizes the resource overhead by remaining idle until invoked again.

V. Evaluation

To show the full potential of our approach, we evaluate the autoscaling platform and RASK empirically. First, we describe the implementations of our prototype and the three processing services embedded. Afterward, we describe the conducted experiments, which analyze the training time of RASK, its performance in comparison with SOTA autoscalers, and its scalability



(a) QR Detector (*QR*) (b) Object Detector (*CV*) (c) Lidar Renderer (*PC*)

Fig. 4. Output of the three implemented processing services.

for larger problem sizes. We provide our implementations and experimental results on GitHub[4].

A. Prototype Implementation

We implement MUDAP and RASK both in Python— building heavily on existing tools and packages. For MUDAP, we containerize applications in Docker, which allows dynamically scaling the container limits (e.g., CPU quota) through the Docker API. The REST API itself consists of a simple HTTP server executed within the container; requests to the REST API are either executed over the Docker API, or routed to the respective service. Lastly, we use Prometheus[5] as our time-series DB, which supports a wide range of queries and metrics aggregation. For RASK, we train regression functions between dependent variables using sklearn [51] with a polynomial degree $\delta = 2$. We use scipy [52] as our numerical solver, which internally uses SLSQP [50]—an algorithm that supports bounded parameters and global resource constraints. To capture RASK's overhead, we also wrap it in a container.

B. Service Implementation

We develop three processing services that will be executed within the MUDAP platform. Fig. 4 shows the demo output for each of the three services, namely, we implement: (a) a QR code reader that detects QR code within video frames through OpenCV [53]; (b) a CV service that uses Yolov10 [54] for detecting and classifying objects in video frames; and (c) a point cloud renderer based on the Kitti dataset [55] that draws a mobile map around a moving vehicle.

All three services operate in cycles of 1000 ms—every second, they retrieve data items from the buffer (i.e., either video frames or point cloud binaries), and process as many items as possible. Upon completion, or after exceeding 1000 ms, the number of processed items (i.e., the *throughput*) and other metrics are scraped by the time-series DB.

B. :

a) Elasticity Parameters: For ensuring SLOs, each service supports its own set and range of elasticity parameters. We summarize this in Table II; for example, the QR service features three variables: *cores*, *data quality*, and *completion* rate. While *cores* and *data quality* can be adjusted as elastic parameters,

[4] Github repository with experimental results in subfolder.

[5] Prometheus Time-Series DB, last accessed on July 17, 2026.

TABLE II
SERVICE VARIABLES FOR THE QR, CV, AND PC SERVICE; SLO TARGETS AND THEIR WEIGHTS (I.E., IMPORTANCE) INCLUDED

Service	Variable	Descript.	Range	SLO	w	Step
QR	*cores*	CPU quota	$(0, 8)$	–	–	float
	data quality	Image size	$[10^2, 10^3]$	≥ 800	0.5	± 1
	completion	Rate finish	$[0, 1]$	≥ 1.0	1.0	–
CV	*cores*	CPU quota	$(0, 8)$	–	–	float
	data quality	Image size	$[128, 320]$	≥ 288	0.2	± 32
	model size	Yolov10[n/.]	$[1, 4]$	≥ 3	0.2	± 1
	completion	Rate finish	$[0, 1]$	≥ 1	1.0	–
PC	*cores*	CPU quota	$(0, 8)$	–	–	float
	data quality	Lidar range	$[6, 60]$	≥ 40	0.5	± 1
	completion	Rate finish	$[0, 1]$	≥ 1	1.0	–

completion is calculated as

$$completion = throughput \ / \ RPS \tag{6}$$

While all three services can adjust their *data quality*, the parameter refers to different aspects and different ranges: for QR and CV, it represents the size of the video frame, while for PC it defines the maximum range of processed points. Notably, rescaling *data quality* causes no additional delay. The CV service exposes one more parameter—*model size*—which allows switching from YOLOv10n up to v8l, corresponding to the range [1,4]. This change is performed at runtime by loading the new model into memory and swapping the ONNX model execution. To support variable input size (i.e., *data quality*), we export Yolov10 with the *dynamic* preset. Still, the size of input tensors must be a multiple of 32—otherwise, the service API clips it to the next valid assignment.

As discussed before, the scaling agent builds regression functions between dependent variables—expressed through structural knowledge (K). Given the variables in Table II, we define K for the three services as

$$\begin{aligned} K_{QR}, K_{PC} &= \{cores, data\ quality\} \rightarrow tp_{\max} \\ K_{CV} &= \{cores, data, model\ quality\} \rightarrow tp_{\max} \end{aligned} \tag{7}$$

which expresses the maximum expected throughput ($tp_{\max}$) according to the service configuration. The $tp_{\max}$ is calculated independently of the current RPS, solely using the processing *latency*. When building the regression model, this allows extrapolating from known configurations to potentially higher RPS; thus answering the question: *could I have processed more data with the current configuration*? Still, the solver uses the $tp_{\max}$ equally for calculating the *completion* SLO.

b) Service Level Objectives: To ensure QoE, all services aim for high *data quality*. In particular, this means high video resolutions for the QR service (i.e., ≥ 800 px) and the CV service (i.e., ≥ 288 px). Apart from that, the CV service aims for high detection accuracy through a large model size (i.e., $\geq$ v8m). Further, all services aim to fulfill 100% completion rate, as known from (6)—this means, that services must process all items stored in the buffer before frames can time out after 1000 ms. While there is no SLO target on the allocated *cores*,

TABLE III
DEFAULT RPS AND ELASTIC PARAMETER ASSIGNMENTS; BETWEEN EXPERIMENTAL RUNS, WE RESET SERVICES TO THESE VALUES

Service	Default RPS	Variable	Value
QR	80	*cores*	2.6
		data quality	550
CV	5	*cores*	2.6
		data quality	224
		model size	3
PC	50	*cores*	2.6
		data quality	30

they impact the service *throughput* (cfr (7)). Hence, the goal is learning a globally-optimal resource division.

A resource-constrained Edge device will fail to reach the desired *throughput* under increasing RPS, particularly when splitting resources among services. To that extent, our scaling agent can trade off qualitative aspects of the application (e.g., *data quality* or *model size*) to sustain a minimum *throughput*—this behavior can be configured through the associated SLO weights. By assigning the highest weight to the *completion* SLO (i.e., $w = 1.0$), we create a hierarchy of qualitative aspects that must be ensured by all means, and such that can be traded off during periods of high load. Additionally, these weights can be tuned to shift importance between services—needed to orchestrate between multiple concurrent tenants.

c) Default Parameter Assignments and Request Load: At the beginning of experiments or an experimental run, we reset the processing services (i.e., all their elasticity parameters) to the default states shown in Table III. Initially, each service is allocated a equal share from the device resources: $C \;/\; |S|$. The other elastic parameters are assigned with the half range of their bounds $\Rightarrow (p^{\max} - p^{\min}) \;/\; 2$. Lastly, we also set the incoming RPS to a default value; this assignment is adjusted at later experiments according to the service load.

C. Experiment Setup & Results

To evaluate the performance and viability of our approach from various angles, we design the following six experiments: **E1** analyzes the convergence of RASK under different hyperparameter settings; **E2** analyzes the effect of different polynomial degrees on the regression functions; **E3** compares the performance of RASK with SOTA baselines. We then analyze the scalability of RASK for (**E4**) increasing numbers of elasticity strategies, (**E5**) the effect of caching parameter assignments, and (**E6**) increasing processing services. For each experiment, we outline the setup of the processing environment, and then provide the respective results.

All experiments were conducted on a VM with an AMD EPYC 7742 CPU and 12 GB of RAM. We prefer this setup over a physical Edge device (e.g., NVIDIA Jetson[6]), because this allows changing the resource constraints between experiments—needed for **E6**. Although RASK is designed to optimize any parameterizable hardware allocation, we focus our evaluation

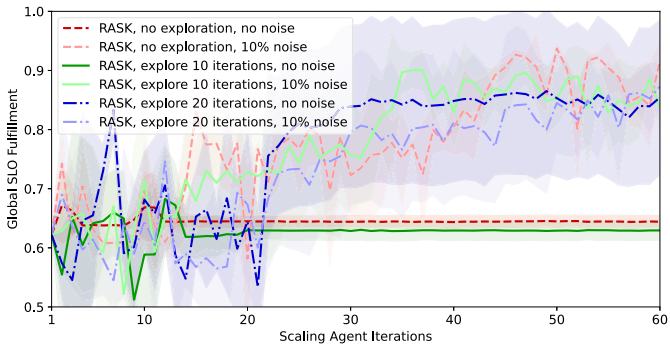


Fig. 5. Training RASK scaling agent in 60 iterations (=10 min); different runs show how training convergence and SLO fulfillment are impacted by an extended initial exploration phase, and Gaussian noise added to the inferred scaling actions.

only on allocating 8 CPU *cores*. To the present day, the GPU cannot be limited for individual Docker containers.[7] Also, we found that all our service configurations had a combined RAM usage of less than 4 GB, which would not exhaust an Edge devices like NVIDIA Jetson. Hence, integrating other hardware dimensions remains for future work.

*1) Training Duration of RASK (**E1**):* The exploration in RASK can be tuned through two hyperparameters: the number of rounds (ξ) in which the agent explores randomly, and the amount of Gaussian noise (η) added to actions. In RL, $\eta = 0.1$ is a common values, whereas we estimate ξ according to previous work [41]. Hence, we define three assignments of $\xi = \{0, 10, 20\}$ and two of $\eta = \{0, 0.1\}$, and use their product for evaluating 6 different hyperparameter combination. Each configuration is executed for 10 min and the agent operates in cycles of 10 s; hence, 60 iterations with the environment. To stabilize our findings, we execute 5 repetitions per config.

a) Results: For each combination of hyperparameters, Fig. 5 shows the globally-weighted SLO fulfillment averaged over all three services, which we calculate according to

$$\left(\sum_{i=1}^{S} \left(\sum_{j=1}^{Q_i} \phi_j \times w_j \right) \Big/ \sum_{j=1}^{Q_i} w_j \right) \Big/ |S| \tag{8}$$

We observe three things: (1) without a dedicated exploration phase $\{\xi = 0\}$, or merely a short one $\{\xi = 10\}$, the agent did not find satisfying parameter assignments. Nevertheless, (2) all noisy configurations with $\{\eta = 0.1\}$ led to satisfying assignments; however, their high fluctuation, even in late iterations, indicates that the noise should decay as the performance converges. Finally, (3) exploring merely for 20 iterations appears sufficient for developing a stable regression model; however, the high standard deviation also indicates that in one run it could not find a highly-satisfying assignment.

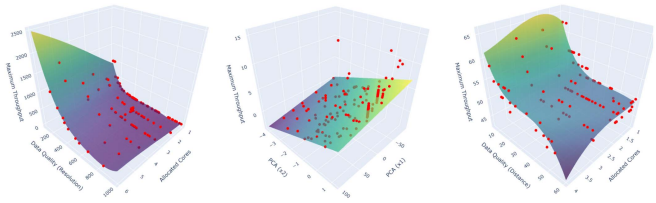
Notice, that all remaining experiments will use the metrics of **E1** $\{\xi = 20, \eta = 0\}$ for pretraining the RASK agent.

[6] NVIDIA Jetson Documentation, last accessed on July 17, 2026.

[7] Docker Limits without GPU partitioning, last accessed July 17, 2026.

TABLE IV
MODEL ACCURACY FOR FITTING A REGRESSION FUNCTION TO THE TRAINING DATA CREATED IN **E1**. CELLS SHOW THE MEAN SQUARED ERROR (MSE) ON A 20% TEST SPLIT; GREEN (BRIGHT) CELLS INDICATE THE OPTIMAL POLYNOMIAL DEGREE FOR FITTING THE FUNCTION, AND ORANGE (DARK) CELLS THE DEFAULT SETTING IN OUR EXPERIMENTS.

Poly. Degree	QR Detector	CV Analyzer	PC Visualizer
1	19114.41	3.65	3.85
2	6315.82	4.22	2.53
3	4290.81	4.64	2.27
4	2650.09	4.25	2.17
5	4168.13	4.98	2.23
6	5740.43	4.66	2.28



(a) QR Detector (*QR*) (b) Object Detector (*CV*) (c) Lidar Renderer (*PC*)

Fig. 6. Structural knowledge of the expected service *throughput* under different service configurations; the degree (δ) of the polynomial function is determined according to the least MSE from Table IV: QR and PC use $\delta = 4$, and CV $\delta = 1$.

*2) Polynomial Degree of RASK (**E2**):* While we started our evaluations using a default polynomial degree of $\delta = 2$, we hypothesized that a service-specific degree might improve the generalization to unseen data. To that extent, we analyze the impact of different polynomial degrees on the mean squared error (MSE) of a 20% test split; for this, we use exclusively training data from **E1**. This analysis also mitigates the risk of overfitting when using high polynomial degrees.

a) Results: For each service, Table IV shows the MSE for using polynomial degrees from 1 to 6: QR and PC are best expressed through a polynomial function of $\delta = 4$, while CV best uses a linear function (i.e., $\delta = 1$). In particular, the CV service decreased its MSE by a factor of **2.4** compared to the default degree. Since our expert knowledge of the structure K did not include the optimal function degree, we see strong evidence for using service-specific degrees in future work.

Further, we visualizing the regression models using each service's optimal degree. In Fig. 6 we show the expected *throughput* (i.e., the $tp_{\max}$) for different parameters assignments. The RASK agent will consider these dynamics to build a tradeoff between parameters. While QR and PC only feature two parameters—making them easily displayed in 3D—the CV also allows adjusting the Yolov10 *model size*, so we reduce its features through PGA. We note that the service *throughput* is always highly impacted by both *data quality* and *cores*. An exception here is the PC service, which shows little change on the *throughput* for increased/decreased *resources*. Connecting to the core assumptions of this paper—that an individual service benefits from multi-threading—this highlights poor parallelization to make use of the additional resources.

*3) Performance Comparison for Dynamic Workloads (**E3**):* To analyze the performance of RASK further, we compare it to two baseline agents. During the experiments, agents will be evaluated according to their capability to ensure SLO fulfillment during two dynamic and unseen request pattern.

a) Baselines: Common autoscalers often use the default k8s implementation, or algorithms based on RL [19], [33]. To that extent, we compare the performance of RASK against the following two baselines:

VPA – Replicates the behavior of the k8s VPA. For each service container, it aims to maintain a resource slack of 5% to 15% [45]; hence, it consumes between 85% and 95% of the maximum scheduled CPU quota. If the service crosses these bounds, the **VPA** agent adjusts the allocated $cores \pm 0.25$. If all available resources are allocated, they can only be reassigned once released.

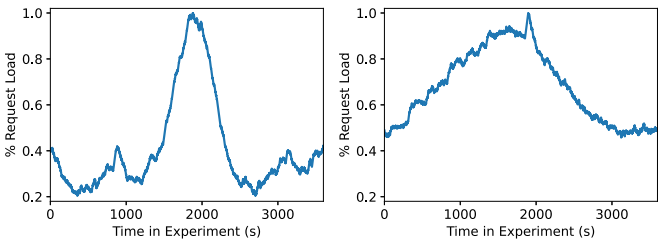
DQN – Approximates Q-values for discrete state-action pairs. To support service-specific scaling policies, services are modeled through separate Deep Q-Networks (DQNs) [11], [13], [15]. Models are pre-trained jointly within a shared Gymnasium[8] environment: given an action, the environment estimates the expected state and reward (i.e., SLO fulfillment); for this, it reuses RASK's regression model. The **DQN** agent has access to all available elasticity dimensions; however, to simplify the action space, it infers a single action per service.

The DQN and the RASK agent are both pretrained on one static request pattern—the default RPS in Table III; for this, the agents can only use the initial observations from **E1**. While the baseline agents scale services according to their own internal logic, they likewise operate on the MUDAP platform: all agents query service states from the time-series DB and adjust elasticity parameters through the REST API. Also, all agents equally start the service execution from the default values in Table III. Given that the VPA agent cannot adjust *data quality*, this makes it technically impossible to meet the associated SLO and maximize fulfillment; however, our experiments showed that the *CV* service would otherwise not reach a $throughput \geq 1$ fps, and hence perform even worse.

b) Dataset: We evaluate the agents' performance using two common request patterns from the Google Cluster production environments [56], [57]. These traces exhibit dynamically changing user behavior and unprecedented bursts—forming a drastic change compared to the static, pretrained pattern. Fig. 7 visualizes both request pattern—each capturing a duration of one hour. Notice, how the relative request load is scaled to a maximum of 100 RPS for the *QR* service and 10 RPS for our *CV* service. Hence, it becomes increasingly more difficult to fulfill the SLOs from Table II. For the PC service, however, we assume a constant RPS—reasonable for an individual vehicle client. We run 5 repetitions for each evaluated agent.

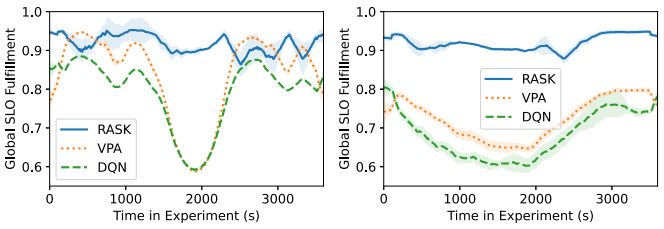
c) Results: Fig. 8 shows the mean globally-weighted SLO fulfillment for the request pattern, including the standard deviation—we added the detailed SLO fulfillment and resource

[8] Gymnasium Env for pretraining RL agents, last accessed July 17, 2026.



(a) Bursty (b) Diurnal

Fig. 7. Two common request patterns in Google Cluster production environments [56], [57]. We scale the relative request load to a maximum of **100** RPS for the *QR* service and **10** RPS for the *CV* service. Each agent is evaluated under both load patterns in separate experiments, each lasting 1 h.



(a) Bursty (b) Diurnal

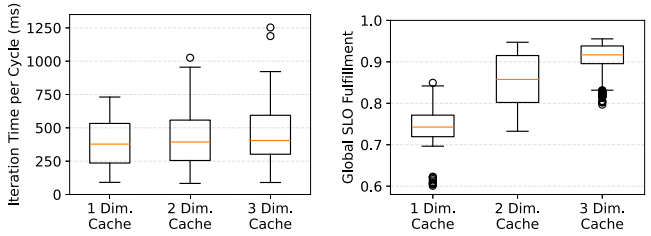
Fig. 8. Global SLO fulfillment across all three services during the *Bursty* and *Diurnal* request patterns. The resulting performance closely aligns with the peaks in the request load, which our method (i.e., RASK) sustains with the fewest violations.

allocation as Appendices. We observe: (1) in periods of high load (i.e., load ≥ 0.4) there is a particularly large gap between RASK and the baselines; while (2) in periods of lower load (i.e., load < 0.4) the agents reported comparable SLO fulfillment, with VPA even outperforming RASK at load $= 0.2$. While the VPA's low SLO fulfillment supports our push towards multiple elasticity strategies, the DQN's low fulfillment requires additional analysis. To date, we attribute it to the fact that the DQN was not trained for different RPS, but neither was the RASK agent in **E1**. Hence, we conclude that RASK generalizes better under these circumstances.

*4) Complexity of RASK – Elasticity Challenges (**E4**):* Our final three experiments assess the complexity and scalability of our approach for increasingly larger problem sizes. We compare three instances of our RASK agent: operating in 1 dimension—adjusting only *cores*; in 2 dimensions—adjusting *cores* and *data quality*; and in 3 dimensions—adding *model size* for the CV services. We evaluate these agents for the *Diurnal* request pattern—5 repetitions each—and capture RASK's overhead as in [58]: CPU, RAM, and performed GFLOPS.[9]

a) Results: Fig. 9 shows the distributions of the RASK agent's runtime (ms) for inferring a parameter assignment, together with the SLO fulfillment throughout the runs. Considering these plots, we observe that increasing the number of elasticity strategies improved the global SLO fulfillment from

[9] To count GFLOPS per scaling cycle, we capture the RASK container's `fp_ret_sse_avx_ops.all` and divide by the # of performed cycles.



(a) Runtime Complexity (b) SLO Fulfillment

Fig. 9. Runtime complexity and SLO fulfillment of RASK with increasing numbers of elasticity strategies (i.e., **Dim**ensions). Agents are evaluated on the *Diurnal* request pattern.

TABLE V
LATENCY BREAKDOWN FOR RASK AT RUNTIME: DURING REGULAR OPERATION MODEL TRAINING DOMINATES THE OVERALL LATENCY; DURING EXPLORATION ONLY RANDOM ASSIGNMENTS ARE INFERRED

Phase	Train model	Infer params.	Orchestrate	Total
Explore	—	< 1 ms (2 %)	10 ms (98%)	10 ms
Exploit	355 ms (90 %)	30 ms (7 %)	10 ms (3 %)	395 ms

TABLE VI
RUNTIME OVERHEAD OF RASK AUTOSCALER: HIGHER ELASTICITY DIMENSIONS MILDLY INCREASE THE CPU DEMAND AND THE AVERAGE GFLOPS PERFORMED IN EVERY SCALING CYCLE

Setup	Occupied Permanently CPU	RAM	Average per Cycle GFLOPS
1 Dim.	0.024 cores	105 MB	0.062 GFLOPS
2 Dim.	0.033 cores	105 MB	0.090 GFLOPS
3 Dim.	0.043 cores	107 MB	0.135 GFLOPS

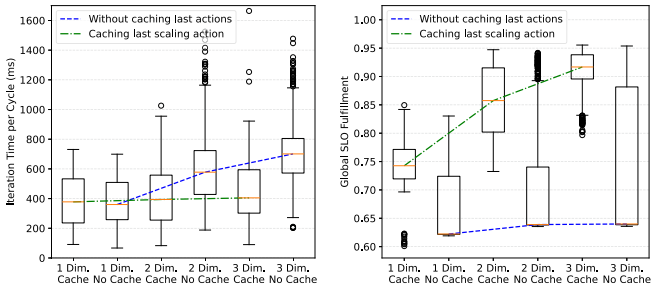
a median value of $\mathbf{0.75}$ for 1 dimension up to $\mathbf{0.92}$ for 3 dimensions. Further, regarding the agent's runtime, we report a minor increase between a median of $\mathbf{357\ ms}$ for 1 dimension compared to $\mathbf{395\ ms}$ for 3 dimensions. In comparison, baseline agents required on average less than $\mathbf{50\ ms}$.

To analyze the runtime complexity of RASK's individual steps (i.e., training, inference, and orchestration) we create an average latency breakdown in Table V: for 3 dimensions, the agent's latency is dominated by the training part ($\mathbf{90}\%$). As a comparison, we analyze the latency during the exploration phase in **E1**, where the agent does not yet train regression models, but infers random parameter assignments.

Finally, Table VI, presents the resource overhead introduced by the the RASK agent: while the permanently occupied RAM stays constant across increasing elasticity dimensions, the CPU usage and performed FLOPS scale with the dimension.

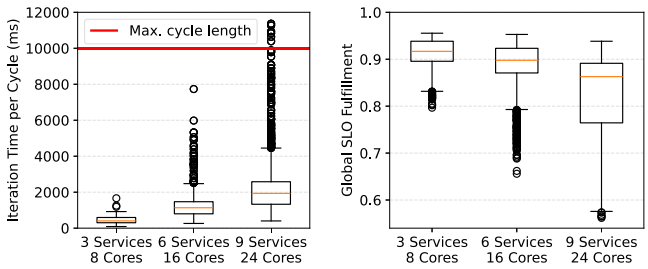
*5) Caching of RASK Assignments (**E5**):* In Section IV-B3 we stated our concerns about how caching parameter assignments between RASK iterations might trap the numerical solver in a local optimum. To that extent, we use the results of **E4**—the three RASK agents with increasingly more elasticity strategies—and compare it to three agents without caching.

a) Results: Fig. 10 shows the runtime and SLO fulfillment for RASK agents with caching enabled or disabled. Considering these plots, we reason that (1) caching the last parameter



(a) Runtime Complexity (b) SLO Fulfillment

Fig. 10. Runtime complexity and SLO fulfillment of RASK with / without caching last parameter assignment. Agent was evaluated on *Diurnal* pattern with different **Dim**ensions.



(a) Runtime Complexity (b) SLO Fulfillment

Fig. 11. Runtime complexity and SLO fulfillment of RASK for increasing numbers of processing services under equivalent resource constraints. Agent faces *Diurnal* request pattern.

assignment ensured stable runtime $\leq$ **400 ms**, regardless of increasing dimensionality. The non-caching agents, however, ran into a mean runtime of **721 ms** for 3 dimensions. Also, (2) caching did show no sign of compromising the SLO fulfillment; on the contrary, from Fig. 10(b) we observe that it stabilized the SLO fulfillment towards more satisfying assignments: for 3 dimensions, the caching agent outperformed the non-caching agent by **32**% SLO fulfillment.

*6) Scalability of RASK – Processing Services (**E6**):* The number of processing services naturally dictates the complexity of optimizing their parameter assignments. Suppose our approach works scale-free, it could ensure equal SLO fulfillment for a larger number of services under analog resource restrictions. We evaluate the RASK agent in 3 different environments: the default setup with 3 services and maximum cores $c_{\max} = 8$; then 6 services with $c_{\max} = 16$; and finally 9 services with $c_{\max} = 24$. Hence, the maximum number of available *cores* grows proportionally to the services.

To avoid implementing six additional service types, we replicate the existing three services (i.e., QR, CV, and PC) and spawn up to three containers for each image. However, from an agents perspective, different instances of the same service are treated completely independent at inference time. Hence, the agent uses the same regression function for up to three QR services, but infers independent parameter assignments for each. Thus, the solver has to optimize $|P| = \{7, 14, 21\}$ parameters for $|S| = \{3, 6, 9\}$ services respectively. Again, we evaluate the agent under the *Diurnal* pattern and stabilize results by running 5 repetitions per instance.

a) Results: Fig. 11 shows again the distributions of the agent's runtime (ms) and the respective global SLO fulfillment—this time averaging up to 9 services. We notice that: (1) the runtime increases linearly with the numbers of services, leading to a median runtime of **2 s** for 9 services; however, even when caching the last parameter assignment, RASK occasionally (i.e., in the outliers) showed spikes of runtimes larger than **10 s**—thus delaying the autoscaling interval. Also, (2) despite the larger resource constraints, the SLO fulfillment slightly declined with increasing numbers of services: 9 services produced a median fulfillment of **0.87** over all 5 runs. We will carefully interpret this behavior in Section VI, where we discuss the natural limitations of the numerical solver and potential strategies to solve this.

VI. Discussion

In the following, we extract a series of implications from the presented results and use them to answer the posed challenges.

C1: Our results support a shift to multi-dimensional elasticity: **E4** showed how controlling more elasticity dimensions increases the SLO fulfillment of RASK almost linearly (cfr. Fig. 9(b)). RASK achieved this by analyzing the impact of parameter interventions on the service performance (cfr. Fig. 6); also, the SLO weights (cfr. Table II) allowed customizing how the agent builds tradeoffs between parameters. Meanwhile, RASK's operational overhead was low: **0.043** CPU cores for optimizing 9 services (cfr. Table VI). To decrease RASK's runtime latency, we observe that caching the last result accelerates the numerical solver significantly, while simultaneously improving SLO fulfillment (cfr. Fig. 10). Still, when analyzing the scalability in **E6**, we observe that higher numbers of services ($|S| \geq 9$) face increased runtime and SLO violations under proportional resource constraints (cfr. Fig. 11). This shows the natural limitations of the numerical solver used, to which we see numerous solutions: (1) detecting similarities (e.g., profiles) between services—in our case we assumed completely independent services; (2) offloading the autoscaling task to a remote node with more resources; (3) parallelizing the autoscaling task between multiple agents to split overall complexity; (4) taking advantage of RASK's containerization and allocate more resources; and (5) decreasing the model training frequency or run in a detached thread—Table V showed that this dominated overall latency.

Implication: Multi-dimensional autoscaling improves the SLO fulfillment on resource-constrained Edge devices with increasing dimensionality. This creates a complex optimization problem that requires a sophisticated solver.

C2: Our findings underline that RASK can quickly build service-specific scaling policies: **E1** showed that 20 iterations in the environment—corresponding to **200 s** runtime—were sufficient to develop a generalizable model for three heterogeneous processing services (cfr. Fig. 5). During this time, the RASK agent explores each services' action space randomly; this might be accelerated by avoiding visited configurations or assigning a dedicated information gain [41] to certain areas

in the configuration space. This also avoids rare cases (i.e., one run for **E1** $\{\xi = 20, \eta = 0\}$) where the random exploration did not find satisfying parameter assignments. After this short training period, the agent was fully trained for solving the other experiments—including request pattern that it had not experienced before. To increase the accuracy of the regression models, **E2** also showed how a custom polynomial degree per service can decrease the MSE significantly: using the recommended polynomial degree $\delta = 4$ for the QR service reduces the MSE by a factor of **2.4** (cfr. Table IV). RASK required **395 ms** for adjusting 3 services in up to 3 dimensions (cfr. Fig. 9), which includes training the regression models and running the numerical solver. Under these circumstances, we see high benefit and little overhead from learning or adjusting the optimal polynomial degree during service runtime.

Implication: Using few samples, RASK develops an accurate model of the processing environment that supports service-specific scaling policies. Efficient exploration and refinement are needed to ensure model accuracy.

C3: Our results highlight RASK's superiority against SOTA under resource scarcity: **E3** showed that RASK sustains dynamic request pattern with up to 28% higher SLO fulfillment compared to a default k8s VPA and a RL-based autoscaler (cfr. Fig. 8). The performance gap between RASK and the baselines was particularly large during peak load, when RASK was the only service to sustain the SLO fulfillment of the CV service by tuning its qualitative parameters. All the evaluated agents operated on our multi-dimensional autoscaling platform, thus benefiting from vertical scaling without any cold start. However, only the RASK agent operated in a continuous action space (cfr. Fig. 6), which highlights its strengths for fine-grained scaling of CPU quota or data quality. While a potential caveat of our method is the increased development effort for exposing service-specific parameters, we believe that the increased SLO fulfillment pays this off.

Implication: Compared to SOTA, multi-dimensional autoscaling achieves higher SLO fulfillment during periods of high request load. Continuous scaling actions support fine-grained adjustments of services and their resources.

VII. Conclusion

In this work, we introduced MUDAP, a Multi-dimensional Autoscaling Platform tailored for resource-constrained Edge environments, along with RASK, a regression-based scaling agent that enables fine-grained vertical scaling of both resource-level and service-level parameters. Our approach addresses key challenges in modern Edge computing—namely, ensuring SLO fulfillment under strict resource constraints, tailoring elasticity strategies to heterogeneous services, and adapting to dynamic request patterns. We evaluate our approach using three stream processing services: a Yolov10 object detector, a QR code reader, and a Lidar renderer. Our results show that RASK can learn service-specific autoscaling models within less than 20 autoscaling cycles (i.e., 200 s of processing). Compared to Kubernetes VPA and DQN-based baselines, our method achieves up to 28% fewer SLO violations, particularly during periods of high load, while introducing minimal resource overhead. Most notably, we demonstrated how an increasing number of elasticity dimensions leads to higher SLO fulfillment. These contributions pave the way for more responsive and efficient autoscaling mechanisms that go beyond traditional resource-focused strategies.

References

[1] M. De Donno, K. Tange, and N. Dragoni, "Foundations and evolution of modern computing paradigms: Cloud, IoT, edge, and fog," *IEEE Access*, vol. 7, pp. 150936–150948, 2019.

[2] L. Liu, C. Chen, Q. Pei, S. Maharjan, and Y. Zhang, "Vehicular edge computing and networking: A survey," *Mobile Netw. Appl.*, vol. 26, no. 3, pp. 1145–1168, 2021.

[3] P. Dazzi et al., "Urgent edge computing," in *Proc. 4th Workshop Flexible Resour. Appl. Manage. Edge*, 2024, pp. 7–14.

[4] S. Dustdar, Y. Guo, B. Satzger, and H.-L. Truong, "Principles of elastic processes," *IEEE Internet Comput.*, vol. 15, no. 5, pp. 66–71, Sep./Oct. 2011.

[5] K. Labs, May 2023. [Online]. Available: https://kubernetes.io/blog/2023/05/12/in-place-pod-resize-alpha/

[6] Y. Li, Y. Lin, Y. Wang, K. Ye, and C. Xu, "Serverless computing: State-of-the-art, challenges and opportunities," *IEEE Trans. Serv. Comput.*, vol. 16, no. 2, pp. 1522–1539, Mar./Apr. 2023.

[7] R. Gajanin, C. Marcelino, and S. Nastic, "Performance isolation for serverless functions," *IEEE Trans. Serv. Comput.*, vol. 18, no. 6, pp. 4408–4424, Nov./Dec. 2025.

[8] I. Taleb, J.-L. Guillaume, and B. Duthil, "A survey on services placement algorithms in integrated cloud-fog/edge computing," *ACM Comput. Surv.*, vol. 57, no. 11, pp. 1–36, 2025.

[9] S. Nastic et al., "SLOC: Service level objectives for next generation cloud computing," *IEEE Internet Comput.*, vol. 24, no. 3, pp. 39–50, May/Jun. 2020.

[10] H. Qiu, S. S. Banerjee, S. Jha, Z. T. Kalbarczyk, and R. K. Iyer, "FIRM: An intelligent fine-grained resource management framework for SLO-oriented microservices," in *Proc. 14th USENIX Symp. Operating Syst. Des. Implementation*, 2020, pp. 805–825.

[11] S. Xue et al., "A meta reinforcement learning approach for predictive autoscaling in the cloud," in *Proc. 28th ACM SIGKDD Conf. Knowl. Discov. Data Mining*, 2022, pp. 4290–4299.

[12] H. Qiu et al., "AWARE: Automate workload autoscaling with reinforcement learning in production cloud systems," in *Proc. USENIX Annu. Tech. Conf.*, 2023, pp. 387–402.

[13] R. Mayerhofer, A. Morichetta, A. Furutanpey, and S. Dustdar, "HPAQT: Adaptive and interpretable high-level SLO-aware autoscaling with reinforcement learning," in *Proc. 18th IEEE/ACM Int. Conf. Utility Cloud Comput.*, 2025, pp. 1–9.

[14] H. Nayir, A. Demir, and H. Kaya, "Multi-dimensional autoscaling in microservices using reinforcement learning," in *Proc. 33rd Int. Symp. Model., Anal. Simul. Comput. Telecommun. Syst.*, 2025, pp. 1–8.

[15] M. Xu et al., "CoScal: Multifaceted scaling of microservices with reinforcement learning," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 4, pp. 3995–4009, Dec. 2022.

[16] J. Fürst, M. Fadel Argerich, B. Cheng, and A. Papageorgiou, "Elastic services for edge computing," in *Proc. 14th Int. Conf. Netw. Service Manage.*, 2018, pp. 358–362.

[17] F. Romero, Q. Li, N. J. Yadwadkar, and C. Kozyrakis, "INFaaS: Automated model-less inference serving," in *Proc. USENIX Annu. Tech. Conf.*, 2021, pp. 397–411.

[18] B. Sedlak et al., "SLO-aware task offloading within collaborative vehicle platoons," in *Proc. 22nd Int. Conf. Service-Oriented Comput.*, Singapore, Springer Nature, 2024, pp. 72–86.

[19] G. Quattrocchi, E. Incerto, R. Pinciroli, C. Trubiani, and L. Baresi, "Autoscaling solutions for cloud applications under dynamic workloads," *IEEE Trans. Serv. Comput.*, vol. 17, no. 3, pp. 804–820, May/Jun. 2024.

[20] V. Cardellini, P. Dazzi, G. Mencagli, M. Nardelli, and M. Torquati, "Scalable compute continuum," *Future Gener. Comput. Syst.*, vol. 166, May 2025, Art. no. 107697.

[21] M. Xu, L. Wen, J. Liao, H. Wu, K. Ye, and C. Xu, "Auto-scaling approaches for cloud-native applications: A survey and taxonomy," 2025, *arXiv:2507.17128*.
[22] C. Klein, M. Maggio, K.-E. Årzén, and F. Hernández-Rodriguez, "Brownout: Building more robust cloud applications," in *Proc. 36th Int. Conf. Softw. Eng.*, 2014, pp. 700–711.
[23] N. Wang, M. Matthaiou, D. S. Nikolopoulos, and B. Varghese, "DYverse: Dynamic vertical scaling in multi-tenant edge environments," *Future Gener. Comput. Syst.*, vol. 108, pp. 598–612, 2020.
[24] Y. Sharma, D. Bhamare, N. Sastry, B. Javadi, and R. Buyya, "SLA management in intent-driven service management systems: A taxonomy and future directions," *ACM Comput. Surv.*, vol. 55, no. 13s, pp. 292:1–292:38, Jul. 2023.
[25] A. Keller and H. Ludwig, "The WSLA framework: Specifying and monitoring service level agreements for web services," *J. Netw. Syst. Manage.*, vol. 11, no. 1, pp. 57–81, Mar. 2003.
[26] A. Danilenka et al., "Adaptive active inference agents for heterogeneous and lifelong federated learning," 2025. [Online]. Available: https://arxiv.org/abs/2410.09099
[27] T. Pusztai et al., "SLO script: A novel language for implementing complex cloud-native elasticity-driven SLOs," in *Proc. IEEE Int. Conf. Web Serv.*, Chicago, IL, USA, 2021, pp. 21–31.
[28] T. Pusztai et al., "Polaris scheduler: SLO- and topology-aware microservices scheduling at the edge," in *Proc. 15th Int. Conf. Utility Cloud Comput.*, 2022, pp. 61–70.
[29] B. Sedlak, V. Casamayor Pujol, P. K. Donta, and S. Dustdar, "Controlling data gravity and data friction: From metrics to multidimensional elasticity strategies," in *Proc. IEEE Int. Conf. Softw. Serv. Eng.*, Chicago, IL, USA, 2023, pp. 43–49.
[30] R. Buyya, C. S. Yeo, S. Venugopal, J. Broberg, and I. Brandic, "Cloud computing and emerging IT platforms: Vision, hype, and reality for delivering computing as the 5th utility," *Future Gener. Comput. Syst.*, vol. 25, no. 6, pp. 599–616, Jun. 2009.
[31] S. Dustdar, Y. Guo, B. Satzger, and H.-L. Truong, "Principles of elastic processes," *IEEE Internet Comput.*, vol. 15, no. 5, pp. 66–71, Sep./Oct. 2011.
[32] T. Chen, R. Bahsoon, and X. Yao, "A survey and taxonomy of self-aware and self-adaptive cloud autoscaling systems," *ACM Comput. Surv.*, vol. 51, no. 3, pp. 61:1–61:40, Jun. 2018.
[33] Y. Garí, D. A. Monge, E. Pacini, C. Mateos, and C. Garcıá Garino, "Reinforcement learning-based application autoscaling in the cloud: A survey," *Eng. Appl. Artif. Intell.*, vol. 102, 2021, Art. no. 104288.
[34] N. Wang, M. Matthaiou, D. S. Nikolopoulos, and B. Varghese, "DYVERSE: DYnamic VERtical scaling in multi-tenant edge environments," *Future Gener. Comput. Syst.*, vol. 108, pp. 598–612, Jul. 2020.
[35] D. Kimovski et al., "Fog or edge: Where to compute?," *IEEE Internet Comput.*, vol. 25, no. 4, pp. 30–36, Jul./Aug. 2021.
[36] D. Petcu, "Service deployment challenges in cloud-to-edge continuum," *Scalable Comput.: Pract. Experience*, vol. 22, no. 3, pp. 313–320, Nov. 2021.
[37] Y. Han, S. Shen, X. Wang, S. Wang, and V. C. Leung, "Tailored learning-based scheduling for kubernetes-oriented edge-cloud system," in *Proc. IEEE Conf. Comput. Commun.*, 2021, pp. 1–10.
[38] Y. Wu et al., "Intelligent resource allocation scheme for cloud-edge-end framework aided multi-source data stream," *EURASIP J. Adv. Signal Process.*, vol. 2023, May 2023, Art. no. 56.
[39] N. Spring, A. Morichetta, B. Sedlak, and S. Dustdar, "MACH: Multi-agent coordination for RSU-centric handovers," Apr. 2025, *arXiv:2505.07827*.
[40] H. Sun, Q. Li, K. Sha, and Y. Yu, "ElasticEdge: An intelligent elastic edge framework for live video analytics," *IEEE Internet Things J.*, vol. 9, no. 22, pp. 23031–23046, Nov. 2022.
[41] B. Sedlak, V. C. Pujol, P. K. Donta, and S. Dustdar, "Equilibrium in the computing continuum through active inference," *Future Gener. Comput. Syst.*, vol. 160, pp. 92–108, 2024.
[42] B. Sedlak, A. Morichetta, P. Raith, V. C. Pujol, and S. Dustdar, "Towards multi-dimensional elasticity for pervasive stream processing services," in *Proc. IEEE Int. Conf. Pervasive Comput. Commun. Workshops*, 2025, pp. 657–660.
[43] S. Laso, I. Murturi, P. Frangoudis, J. L. Herrera, J. M. Murillo, and S. Dustdar, "A multidimensional elasticity framework for adaptive data analytics management in the computing continuum," in *Proc. IEEE Int. Conf. Commun.*, 2025, pp. 1133–1138.
[44] P. Chen, Y. Qi, and D. Hou, "*CauseInfer*: Automated end-to-end performance diagnosis with hierarchical causality graph in cloud environment," *IEEE Trans. Serv. Comput.*, vol. 12, no. 2, pp. 214–230, Mar./Apr. 2019.
[45] Y. Zhao and A. Uta, "Tiny autoscalers for tiny workloads: Dynamic CPU allocation for serverless functions," in *Proc. IEEE Int. Symp. Cluster, Cloud Internet Comput.*, 2022, pp. 170–179.
[46] M. Xing, H. Mao, and Z. Xiao, "Fast and fine-grained autoscaler for streaming jobs with reinforcement learning," in *Proc. 31st Int. Joint Conf. Artif. Intell.*, 2022, pp. 564–570.
[47] N. K. Kitson, A. C. Constantinou, Z. Guo, Y. Liu, and K. Chobtham, "A survey of Bayesian network structure learning," *Artif. Intell. Rev.*, vol. 56, no. 8, pp. 8721–8814, Aug. 2023.
[48] M. J. Vowels, N. C. Camgoz, and R. Bowden, "D'ya like DAGs? A survey on structure learning and causal discovery," *ACM Comput. Surv.*, vol. 55, Mar. 2021, Art. no. 82.
[49] B. Sedlak, V. C. Pujol, P. K. Donta, and S. Dustdar, "Designing reconfigurable intelligent systems with markov blankets," in *Proc. Int. Conf. Service-Oriented Comput.*, Rome, Italy, Springer Nature, 2023.
[50] D. Kraft, *A Software Package for Sequential Quadratic Programming*, Cologne, Germany: Wiss. Berichtswesen d. DFVLR, 1988.
[51] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," 2018. [Online]. Available: https://arxiv.org/abs/1201.0490
[52] P. Virtanen et al., "SciPy 1.0–Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, no. 3, pp. 261–272, Mar. 2020, *arXiv:1907.10121*.
[53] OpenCV, "OpenCV at 4.9.0," 2024. [Online]. Available: https://github.com/opencv/opencv/tree/4.9.0
[54] A. Wang et al., "YOLOv 10: Real-time end-to-end object detection," Oct. 2024, *arXiv:2405.14458*.
[55] A. Geiger, P. Lenz, and R. Urtasun, "Are we ready for autonomous driving? The KITTI vision benchmark suite," in *Proc. Conf. Comput. Vis. Pattern Recognit.*, 2012, pp. 3354–3361.
[56] J. Wilkes, "Google cluster data–2019 traces," 2020. [Online]. Available: https://github.com/google/cluster-data/tree/master
[57] Z. Wang, P. Li, C.-J. M. Liang, F. Wu, and F. Y. Yan, "Autothrottle: A practical bi-level approach to resource management for SLO-targeted microservices," in *Proc. 21st USENIX Symp. Netw. Syst. Des. Implementation*, 2024, pp. 149–165.
[58] M. T. Usman, H. Khan, I. Rida, and J. Koo, "Lightweight transformer-driven multi-scale trapezoidal attention network for saliency detection," *Eng. Appl. Artif. Intell.*, vol. 155, Sep. 2025, Art. no. 110917.



Boris Sedlak (Member, IEEE) received the BSc degree in media informatics with the University of Applied Sciences, St. Pölten, the MSc degree in software engineering and internet computing from TU Wien, and the PhD degree from TU Wien, in 2025. He is a postdoctoral researcher with the Distributed Systems Group, TU Wien, Austria. His research interests include edge intelligence, causal methods for the computing continuum, and service-oriented computing.



Philipp Raith (Member, IEEE) received the MSc degree from the Technical University of Vienna, Austria, in 2021 with distinction in the field of computer science. He is currently working toward the PhD degree with the Distributed Systems Group in the field of edge computing. His research interests include serverless edge computing, edge intelligence and operations for AI.



Andrea Morichetta received the PhD degree in electrical, electronics, and communication engineering from Politecnico di Torino. He is a postdoctoral researcher with the Distributed Systems Group, TU Wien, specializing in machine learning in and for distributed systems, and edge-to-cloud computing. He has collaborated with leading institutions and industry partners such as Cisco (San Jose, CA, US), and Tsinghua University (Beijing, CN).



Víctor Casamayor Pujol (Member, IEEE) received the PhD degree from Universitat Pompeu Fabra. He is a tenure-track assistant professor with Universitat Pompeu Fabra (UPF), Barcelona, Spain. Prior to that, he worked as a postdoctoral researcher with Distributed Systems Group, TU Vienna. His research interests revolve around self-adaptive methodologies for computing continuum systems, including SLO-based definitions, causal and machine learning inference, and robotics.



Schahram Dustdar (Fellow, IEEE) received the PhD degree in business informatics from the University of Linz, Austria, in 1992. He is a full professor of computer science, focusing on internet technologies, and heads the Distributed Systems Group, TU Wien, Vienna, Austria. He is chairperson of the Informatics Section of the Academia Europaea. He has been an associate editor for *IEEE Transactions on Services Computing*, *ACM Transactions on the Web*, and *ACM Transactions on Internet Technology*. He has received several accolades, including the ACM Distinguished Scientist and IBM Faculty awards.